

Failure (bad key + cost comparison) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Simple Version

```
import openai
from openai import OpenAI

client = OpenAI()

# part 1: bad key
bad_client = OpenAI(api_key="sk-fake-key-123")
try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")

# part 2: cost comparison
short = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Be helpful."},
        {"role": "user", "content": "What's the capital of France?"},
    ],
)

long_prompt = "You are a friendly, detailed assistant. " * 20
long = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": long_prompt},
        {"role": "user", "content": "What's the capital of France?"},
    ],
)
```

```
print("Short prompt tokens:", short.usage.total_tokens)
print("Long prompt tokens:", long.usage.total_tokens)
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version — 2 approaches

Approach 1 — print the raw comparison

```
import openai
from openai import OpenAI

client = OpenAI()

def try_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed – check your API key.")

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = "You are a friendly, detailed, thorough assistant who explains

    short = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": short_prompt}, {"role": "user", "c
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": long_prompt}, {"role": "user", "c
    )

    print(f"Short system prompt: {short.usage.total_tokens} tokens")
    print(f"Long system prompt: {long.usage.total_tokens} tokens")

try_bad_key()
compare_prompt_cost("What's the capital of France?")
```

Approach 2 — same comparison, plus a computed dollar cost

Raw token counts are useful, but a dollar figure is what actually makes the trade-off concrete for a real decision.

```
import openai
from openai import OpenAI

client = OpenAI()

# gpt-4o-mini pricing (per 1M tokens) – check current pricing before using this f
PRICE_PER_1M_INPUT = 0.15
PRICE_PER_1M_OUTPUT = 0.60

def estimate_cost(usage) -> float:
    input_cost = (usage.prompt_tokens / 1_000_000) * PRICE_PER_1M_INPUT
    output_cost = (usage.completion_tokens / 1_000_000) * PRICE_PER_1M_OUTPUT
    return input_cost + output_cost

def try_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed – check your API key.")

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = "You are a friendly, detailed, thorough assistant who explains

    short = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": short_prompt}, {"role": "user", "c
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": long_prompt}, {"role": "user", "c
    )

    print(f"Short system prompt: {short.usage.total_tokens} tokens, ~${estimate_c
    print(f"Long system prompt: {long.usage.total_tokens} tokens, ~${estimate_co
```

```
try_bad_key()  
compare_prompt_cost("What's the capital of France?")
```

Why Approach 2 is what you'd actually ship: a token count is meaningful to you, right now, while you're reading it — but a dollar figure is what you'd actually put in front of a manager, or use to compare two design choices against each other months from now. The single-call cost difference looks tiny here, but multiply it by a million real user requests and a bloated system prompt becomes a real, ongoing expense — which is exactly the kind of thing "why is this feature so expensive" post-mortems turn out to be about.